

9) Network Virtualization – OpenVSwitch and Linux bridge

Aim:

To perform network virtualization in Linux using OpenVswitch and Linux Bridge.

OpenVswitch

OpenVSwitch (OVS) is an open-source, production-quality, multi-layer software switch designed for virtualized environments, particularly in Linux-based hypervisors. It provides virtual network connectivity to virtual machines (VMs) and serves as a foundational component in modern data centers and software-defined networks (SDNs). OVS enables network automation through programmatic extensions like the OpenFlow protocol and the OVSDB management protocol, while also supporting standard network management interfaces such as NetFlow, sFlow, and LACP

Linux bridge

A Linux bridge is a software-based network device that functions like a physical network switch, allowing multiple network interfaces (physical or virtual) to be connected and operate as a single logical network segment. It works at the data link layer (Layer 2), forwarding network traffic based on MAC addresses rather than IP addresses. Linux bridges are crucial for virtual networking, enabling virtual machines and containers to communicate with each other and with the physical network, and are often used to create isolated networks or connect different network segments

PROCEDURE:Linux Bridge:

1. Check the current network configuration.

```
tce@tce-VirtualBox:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:3025:d14d:a8a8:85bc  prefixlen 64  scopeid 0x0<gl
obal>
    inet6 fd17:625c:f037:2:e2cf:7173:1f91:86e2  prefixlen 64  scopeid 0x0<gl
```

2. Install and configure necessary Bridge Utilities

- a. Install bridge-utils

```
tce@tce-VirtualBox:~$ sudo apt install bridge-utils
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  bridge-utils
```

- b. Load loads the bridge kernel module, which is required for bridge functionality.

```
tce@tce-VirtualBox:~$ sudo modprobe bridge
```

- c. Loads the br_netfilter kernel module, which enables netfilter (firewall) rules to be applied to traffic passing through the bridge.

```
tce@tce-VirtualBox:~$ sudo modprobe br_netfilter
```

3. Edit the netplan configuration file and specify the configuration.

```
tce@tce-VirtualBox:~$ sudo nano /etc/netplan/01-netcfg.yaml
Thunderbird Mail /etc/netp
network:
  version: 2
  renderer: networkd      # use systemd-networkd backend
  ethernets:
    eno2:
      dhcp4: no           # disable DHCP
  bridges:
    br-cloud:             # the new bridge name
      interfaces: [eno2]
      addresses: [10.0.2.15/24]  # IP + prefix length (24 = 255.255.255.0)
      gateway4: 10.0.2.255    # your gateway
      nameservers:           # optional DNS
        addresses: [8.8.8.8, 1.1.1.1]
```

4. Apply the new netplan configuration and restart the network.

```
tce@tce-VirtualBox:~$ sudo systemctl restart networking
```

5. Verify the network interfaces and bridges

```
tce@tce-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:28:f1:f0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86157sec preferred_lft 86157sec
    inet6 fd17:625c:f037:2:3025:d14d:a8a8:85bc/64 scope global temporary dynamic
        valid_lft 86203sec preferred_lft 14203sec
    inet6 fd17:625c:f037:2:426e:87aa:1395:25aa/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86203sec preferred_lft 14203sec
    inet6 fe80::f38c:68f3:3669:e893/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: br-cld: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
    link/ether 06:ae:ae:2b:2c:98 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global br-cld
        valid_lft forever preferred_lft forever
```

6. Verify the routing table

```
tce@tce-VirtualBox:~$ route -n
Kernel IP routing table
Destination      Gateway         Genmask        Flags Metric Ref    Use Iface
0.0.0.0          10.0.2.2       0.0.0.0        UG      100    0      0 enp0s3
10.0.2.0         0.0.0.0        255.255.255.0  U       0      0      0 br-cld
10.0.2.0         0.0.0.0        255.255.255.0  U      100    0      0 enp0s3
169.254.0.0      0.0.0.0        255.255.0.0    U      1000   0      0 enp0s3

tce@tce-VirtualBox:~$
tce@tce-VirtualBox:~$ brctl show
bridge name      bridge id      STP enabled    interfaces
br-cld           8000.000000000000    no
```

Alternatively, you can use the following commands to configure the linux bridge:

```
root@tce-VirtualBox:~# brctl addbr br-cld-new
root@tce-VirtualBox:~# brctl addif br-cld-new enp0s3
root@tce-VirtualBox:~# ifconfig enp0s3 0
root@tce-VirtualBox:~# ifconfig br-cld-new 10.0.2.15 netmask 255.255.255.0
root@tce-VirtualBox:~# route add default gw 10.0.2.255 dev br-cld-new
SIOCADDRT: Network is unreachable
root@tce-VirtualBox:~# route add default gw 10.0.2.15 dev br-cld-new
root@tce-VirtualBox:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br-cld state UP group default qlen 1000
    link/ether 08:00:27:28:f1:f0 brd ff:ff:ff:ff:ff:ff
    inet6 fd17:625c:f037:2:3025:d14d:a8a8:85bc/64 scope global temporary dynamic
        valid_lft 86230sec preferred_lft 14230sec
    inet6 fd17:625c:f037:2:426e:87aa:1395:25aa/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86230sec preferred_lft 14230sec
    inet6 fe80::f38c:68f3:3669:e893/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: br-cld: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 08:00:27:28:f1:f0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global br-cld
        valid_lft forever preferred_lft forever
4: br-cld-new: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
    link/ether ea:7a:f8:b1:b6:16 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global br-cld-new
        valid_lft forever preferred_lft forever
    inet6 fe80::e87a:f8ff:feb1:b616/64 scope link
        valid_lft forever preferred_lft forever
root@tce-VirtualBox:~# route -n
Kernel IP routing table
Destination      Gateway         Genmask        Flags Metric Ref    Use Iface
0.0.0.0          10.0.2.15      0.0.0.0        UG      0      0      0 br-cld-new
10.0.2.0         0.0.0.0        255.255.255.0  U       0      0      0 br-cld
10.0.2.0         0.0.0.0        255.255.255.0  U       0      0      0 br-cld-new
169.254.0.0      0.0.0.0        255.255.0.0    U      1000   0      0 br-cld-new
root@tce-VirtualBox:~# brctl show
bridge name      bridge id      STP enabled    interfaces
br-cld           8000.00002728f1f0    no             enp0s3
br-cld-new       8000.000000000000    no
```

Open Virtual Switch (OpenVSwitch/OVS):

DATE:

2303917720522176-Yazhini S B

1. Enter as root user.

```
osboxes@osboxes:~$ sudo -s
[sudo] password for osboxes:
root@osboxes:/home/osboxes#
```

2. Create 2 namespace (in my case VRF1 and VRF2)

```
root@osboxes:/home/osboxes# ip netns add VRF1
root@osboxes:/home/osboxes# ip netns add VRF2
root@osboxes:/home/osboxes# ip netns list
VRF1
VRF2
root@osboxes:/home/osboxes#
```

3. Create 4 Virtual ethernet port to attach with bridge and namespaces.

```
root@osboxes:/home/osboxes# ip link add veth1 type veth peer name veth2
```

Similarly add veth3 and veth4.

4. If the configuration is done to create the topology shown in the image, the links will be as

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:b4:19:73 brd ff:ff:ff:ff:ff:ff
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:b5:1f:47 brd ff:ff:ff:ff:ff:ff
4: veth2@veth1: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether b2:4f:5a:97:79:d6 brd ff:ff:ff:ff:ff:ff
5: veth1@veth2: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether b6:da:58:fa:3d:e9 brd ff:ff:ff:ff:ff:ff
6: veth4@veth3: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether 1e:41:04:f5:6d:ed brd ff:ff:ff:ff:ff:ff
7: veth3@veth4: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether 72:3f:6e:37:55:13 brd ff:ff:ff:ff:ff:ff
```

5. Assign each virtual ethernet port to the particular namespace

```
root@osboxes:/home/osboxes# ip link set veth1 netns VRF1
root@osboxes:/home/osboxes# ip link set veth3 netns VRF2
root@osboxes:/home/osboxes# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:b4:19:73 brd ff:ff:ff:ff:ff:ff
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:b5:1f:47 brd ff:ff:ff:ff:ff:ff
4: veth2@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether b2:4f:5a:97:79:d6 brd ff:ff:ff:ff:ff:ff link-netns VRF1
6: veth4@if7: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether 1e:41:04:f5:6d:ed brd ff:ff:ff:ff:ff:ff link-netns VRF2
```

```
root@osboxes:/home/osboxes# ip netns exec VRF1 ip link
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5: veth1@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether b6:da:58:fa:3d:e9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

You can see that veth1 port is attached to the VRF1 namespace.

6. Assign IP addresses to the virtual ethernet ports

```

root@osboxes:/home/osboxes# ip netns exec VRF1 ifconfig veth1 10.10.10.1/24 up
root@osboxes:/home/osboxes# ip netns exec VRF2 ifconfig veth3 10.10.10.2/24 up
root@osboxes:/home/osboxes# ip netns exec VRF2 ifconfig
veth3: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 10.10.10.2 netmask 255.255.255.0 broadcast 10.10.10.255
    ether 72:3f:6e:37:55:13 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@osboxes:/home/osboxes# ip netns exec VRF1 ifconfig
veth1: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 10.10.10.1 netmask 255.255.255.0 broadcast 10.10.10.255
    ether b6:da:58:fa:3d:e9 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

7. Check the configuration using the default routing table

```

root@osboxes:/home/osboxes# ip netns exec VRF2 route -n
Kernel IP routing table
Destination      Gateway         Genmask        Flags Metric Ref    Use Iface
10.10.10.0       0.0.0.0         255.255.255.0  U        0      0        0 veth3
root@osboxes:/home/osboxes# route -n
Kernel IP routing table
Destination      Gateway         Genmask        Flags Metric Ref    Use Iface
0.0.0.0          10.0.2.2        0.0.0.0         UG        100    0        0 enp0s3
10.0.2.0         0.0.0.0         255.255.255.0  U        100    0        0 enp0s3
root@osboxes:/home/osboxes# ip netns exec VRF1 route -n
Kernel IP routing table
Destination      Gateway         Genmask        Flags Metric Ref    Use Iface
10.10.10.0       0.0.0.0         255.255.255.0  U        0      0        0 veth1

```

8. Install openVswitch and its dependencies

```

root@osboxes:/home/osboxes# sudo apt install openvswitch-switch openvswitch-common -y
Installing:
  openvswitch-common openvswitch-switch

Installing dependencies:
  libevent-2.1-7t64 libunbound8 libxdp1 python3-openvswitch python3-sortedcontainers

Suggested packages:
  openvswitch-doc python-sortedcontainers-doc

```

9. Check the status of the openvswitch service, if off turn it on.

```

root@osboxes:/home/osboxes# ovs-vsctl --version
ovs-vsctl (Open vSwitch) 3.4.0
DB Schema 8.7.0
root@osboxes:/home/osboxes# sudo systemctl status openvswitch-switch
● openvswitch-switch.service - Open vSwitch
   Loaded: loaded (/usr/lib/systemd/system/openvswitch-switch.service; enabled; preset: enabled)
   Active: active (exited) since Sun 2025-09-28 02:45:09 EDT; 1min 28s ago
  Invocation: 963c4b96e24d45079539f4626ca199a0
    Process: 42721 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 42721 (code=exited, status=0/SUCCESS)
    Mem peak: 1.4M
       CPU: 10ms

Sep 28 02:45:09 osboxes systemd[1]: Starting openvswitch-switch.service - Open vSwitch...
Sep 28 02:45:09 osboxes systemd[1]: Finished openvswitch-switch.service - Open vSwitch.

```

10. Add the switch using ovs commandline tools and check the configuration

```

root@osboxes:/home/osboxes# ovs-vsctl add-br vSwitch1
root@osboxes:/home/osboxes# ovs-vsctl show
8c551ce7-a290-43fd-9b76-3207815b6965
    Bridge vSwitch1
        Port vSwitch1
            Interface vSwitch1
                type: internal
            ovs_version: "3.4.0"
root@osboxes:/home/osboxes# ovs-vsctl add-port vSwitch1 veth2
root@osboxes:/home/osboxes# ovs-vsctl add-port vSwitch1 veth4
root@osboxes:/home/osboxes# ovs-vsctl show
8c551ce7-a290-43fd-9b76-3207815b6965
    Bridge vSwitch1
        Port veth4
            Interface veth4
        Port veth2
            Interface veth2
        Port vSwitch1
            Interface vSwitch1
                type: internal
            ovs_version: "3.4.0"

```

11. Turn the virtual ethernet ports up and configure IP for switch check their status

```

root@osboxes:/home/osboxes# ifconfig veth2 up
root@osboxes:/home/osboxes# ifconfig veth4 up
root@osboxes:/home/osboxes# ifconfig vSwitch1 10.10.3/24 up

```

```

veth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::b04f:5aff:fe97:79d6 prefixlen 64 scopeid 0x20<link>
    ether b2:4f:5a:97:79:d6 txqueuelen 1000 (Ethernet)
    RX packets 8 bytes 656 (656.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 27 bytes 3112 (3.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

veth4: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::1c41:4ff:fe5:6ded prefixlen 64 scopeid 0x20<link>
    ether 1e:41:04:f5:6d:ed txqueuelen 1000 (Ethernet)
    RX packets 7 bytes 586 (586.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 20 bytes 2383 (2.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

12. Ping the namespace at the other end, if the configuration is done correct we can ping the Virtual namespace 2 from virtual namespace 1 via the switch (ovs bridge)

```
root@osboxes:/home/osboxes# ip netns exec VRF1 ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=0.768 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=0.122 ms
64 bytes from 10.10.10.2: icmp_seq=3 ttl=64 time=0.631 ms
64 bytes from 10.10.10.2: icmp_seq=4 ttl=64 time=0.154 ms
64 bytes from 10.10.10.2: icmp_seq=5 ttl=64 time=0.137 ms
64 bytes from 10.10.10.2: icmp_seq=6 ttl=64 time=0.115 ms
^C
--- 10.10.10.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5938ms
rtt min/avg/max/mdev = 0.115/0.321/0.768/0.270 ms
```

13. Optionally you can turn on the spanning tree protocol (to prevent broadcast storms and network loops)

```
root@osboxes:/home/osboxes# ovs-vsctl set bridge vSwitch1 stp_enable=true
root@osboxes:/home/osboxes# ovsdb-client dump
```

```
Port table
_uuid          bond_active_slave bond_downdelay bond_fake_iface bond_mode bond_updelay cvlans exte
rnal_ids fake_bridge interfaces          lacp mac name      other_config protected qos rstp_statistics
rstp_status statistics
tag trunks vlan_mode
-----
ca82d3a9-b93c-4072-96fc-70bcae8f5111 [] 0 false [] 0 [] {}
{} false [84e0e1f0-e30c-4b3b-b182-5ec37a6a2c1f] [] [] vSwitch1 {} false [] {}
{} [] [] [] {}
941ad466-57bb-4ce5-9437-3d52332ea562 [] 0 false [] 0 [] {}
{} false [51b29bb4-4fc2-4496-866e-d5a04b8d5cc0] [] [] veth2 {} false [] {}
{} {stp_error_count=0, stp_rx_count=0, stp_tx_count=15} {stp_port_id="32770", stp_role=designated, stp_sec_in_
state="0", stp_state=forwarding} [] [] []
40729c0d-1699-4b46-bc18-4f1da21a18d7 [] 0 false [] 0 [] {}
{} false [c0021f2a-32f5-4093-a387-7ba30516b22c] [] [] veth4 {} false [] {}
{} {stp_error_count=0, stp_rx_count=0, stp_tx_count=15} {stp_port_id="32769", stp_role=designated, stp_sec_in_
state="0", stp_state=forwarding} [] [] []
```

14. Finally check the MAC address attached with the switch and the actual mac address of the virtual namespace device, if they match everything is done right.

DATE:

2303917720522176-Yazhini S B

```
root@osboxes:/home/osboxes# ovs-appctl fdb/show vSwitch1
port  VLAN  MAC  Age
  1    0  b6:da:58:fa:3d:e9  119
  2    0  72:3f:6e:37:55:13  118
LOCAL  0  de:63:e8:9f:3b:4f    3
root@osboxes:/home/osboxes# ip netns exec VRF2 ifconfig
veth3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 10.10.10.2  netmask 255.255.255.0  broadcast 10.10.10.255
    inet6 fe80::703f:6eff:fe37:5513  prefixlen 64  scopeid 0x20<link>
    ether 72:3f:6e:37:55:13  txqueuelen 1000  (Ethernet)
    RX packets 467  bytes 29925 (29.9 KB)
    RX errors 0  dropped 368  overruns 0  frame 0
    TX packets 28  bytes 2224 (2.2 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@osboxes:/home/osboxes# ip netns exec VRF1 ifconfig
veth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 10.10.10.1  netmask 255.255.255.0  broadcast 10.10.10.255
    inet6 fe80::b4da:58ff:fe37:5513  prefixlen 64  scopeid 0x20<link>
    ether b6:da:58:fa:3d:e9  txqueuelen 1000  (Ethernet)
    RX packets 483  bytes 31175 (31.1 KB)
    RX errors 0  dropped 375  overruns 0  frame 0
    TX packets 28  bytes 2224 (2.2 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Result:

Thus, we have performed OpenVswitch and Linux Bridge successfully.